

HOUSE PRICE PREDICTION SYSTEM

*A Machine Learning Approach to Real Estate Valuation using the
California Housing Dataset*

AIML Summer Internship 2026

IIHMF, MNNIT Allahabad, Prayagraj

Submitted by

Shanya Kakkar

B.Tech Computer Science & Engineering
United College of Engineering and Research, Prayagraj

Contents

1	Introduction	3
1.1	Background	3
1.2	Problem Statement	3
1.3	Objectives	3
1.4	Scope of the Project	4
1.5	Tools and Technologies Used	4
2	Literature Review	5
2.1	Existing Approaches	5
2.2	Research Studies	5
2.3	Research Gap	6
2.4	Comparative Summary of Related Work	6
3	Methodology	8
3.1	Dataset Description	8
3.2	Data Preprocessing	9
3.3	Algorithms Used	9
3.4	System Workflow	10
4	Implementation	11
4.1	Exploratory Data Analysis (EDA)	11
4.2	Feature Engineering	13
4.3	Model Development	13
4.4	Deployment Architecture	13
4.5	Challenges Faced	14
5	Results and Discussion	15
5.1	Performance Metrics	15
5.2	Comparative Analysis	16
5.3	Error Analysis	18
5.4	Limitations	18
6	Conclusion and Future Scope	19
6.1	Conclusion	19

6.2	Future Scope	19
-----	------------------------	----

Chapter 1

Introduction

1.1 Background

Real estate valuation is one of the most economically significant applications of predictive analytics. The price of a residential property is influenced by a complex interaction of factors including its geographic location, physical characteristics, neighbourhood demographics, and broader market conditions. Traditionally, house valuation has relied on manual appraisal methods conducted by real estate professionals, which are time-consuming, subjective, and prone to inconsistency across appraisers.

With the growth of Machine Learning (ML), it has become possible to build data-driven models that learn pricing patterns directly from historical transaction data. Such models can process a far larger number of variables than a human appraiser and can identify non-linear relationships between property attributes and sale price that are difficult to detect manually. This project applies supervised regression techniques to the California Housing dataset to build an automated house price prediction system.

1.2 Problem Statement

Given a set of measurable attributes of a residential block — including its geographic coordinates, proximity to the ocean, the age of housing structures, and structural indicators such as the number of rooms and households — the goal is to predict the median house value for that block. The core challenge lies in selecting a feature set that is both predictive and interpretable, while avoiding target leakage and maintaining a meaningful mapping to real-world housing characteristics such as area, location, number of rooms, age of property, and available amenities.

1.3 Objectives

- To collect and preprocess a real-world housing dataset (California Housing Prices, Kaggle) suitable for regression modelling.

- To perform exploratory data analysis (EDA) to understand feature distributions, relationships, and the strongest predictors of house price.
- To engineer meaningful derived features (e.g., rooms per household, population per household) that improve model interpretability and accuracy.
- To train and compare at least three regression algorithms — Linear Regression, Random Forest, and XGBoost — using standard evaluation metrics.
- To deploy the best-performing model as an interactive web application using Streamlit, allowing users to input property details and receive a real-time price estimate.

1.4 Scope of the Project

The scope of this project is limited to predictive modelling of median house value at the census-block level using the California Housing dataset; it does not extend to predicting prices for individual, uniquely identified properties, nor does it incorporate live real estate market feeds. The project covers the complete machine learning lifecycle — from raw data ingestion through to a deployed, interactive prediction interface — but does not include hyperparameter optimisation beyond manual tuning, nor does it attempt to source additional external datasets (e.g., crime statistics, school ratings) that were outside the scope of the internship timeline.

The project is intended to demonstrate practical proficiency in the end-to-end machine learning workflow expected of a data science or machine learning engineering role, with particular emphasis on regression modelling, feature engineering trade-off analysis, and model deployment — skills directly applicable to fintech and analytics roles.

1.5 Tools and Technologies Used

Table 1.1: Summary of tools and technologies used in this project

Category	Tools / Libraries
Programming Language	Python 3.10
Data Handling	pandas, NumPy
Visualization	Matplotlib, Seaborn, Plotly
Machine Learning	scikit-learn, XGBoost
Model Persistence	joblib
Web Deployment	Streamlit
Development Environment	Jupyter Notebook, Anaconda
Version Control	Git

Chapter 2

Literature Review

2.1 Existing Approaches

House price prediction has been studied extensively using both traditional statistical methods and modern machine learning techniques. Traditional approaches, such as the Hedonic Pricing Model, treat a house as a bundle of attributes (location, size, age, amenities) and estimate price as a linear combination of these attributes using Ordinary Least Squares (OLS) regression. While interpretable, hedonic models assume linear relationships and struggle to capture complex, non-linear interactions between features — for example, the way location value changes non-linearly with proximity to economic centres.

More recent work has shifted toward ensemble machine learning methods. Tree-based ensemble models such as Random Forest and Gradient Boosting (including XGBoost) have consistently outperformed linear models in housing price prediction tasks because they can capture non-linear feature interactions and are robust to outliers and skewed distributions, which are common in real estate data (e.g., income and price distributions are typically right-skewed).

2.2 Research Studies

Several benchmark studies using the California Housing dataset — originally derived from the 1990 U.S. Census by Pace and Barry (1997) — have become a standard testbed for regression algorithms in the machine learning community. This dataset is frequently used in academic coursework and industry benchmarking because it combines continuous, skewed, and categorical features in a moderately sized, clean format (20,640 records).

Comparative studies consistently report that gradient boosting methods (XGBoost, LightGBM, Gradient Boosting Regressor) achieve the highest R^2 scores on this dataset, typically in the range of 0.80–0.85 when using the full feature set including median income, and somewhat lower (0.65–0.75) when income-related features are excluded. Random Forest models perform comparably, while plain Linear Regression consistently lags behind

due to its inability to model non-linear geographic effects such as the interaction between location and ocean proximity.

This project builds directly on these findings, applying the same three-model comparison methodology (Linear Regression, Random Forest, XGBoost) to validate and reproduce these known performance patterns on a deliberately feature-constrained version of the dataset.

2.3 Research Gap

While extensive literature exists on maximising predictive accuracy for the California Housing dataset, comparatively little attention is paid in academic benchmarks to the interpretability trade-offs involved in feature selection — most published results simply use every available column to maximise R^2 , without discussing whether each feature is conceptually justified for the stated business problem. This project addresses that gap in a small way by explicitly quantifying the accuracy cost of excluding features (`median_income`, `total_bedrooms`) that, while statistically useful, do not map cleanly onto the requested prediction criteria of area, location, rooms, age, and amenities. This decision-driven approach to feature selection is more representative of real-world, stakeholder-constrained ML projects than the accuracy-maximising benchmarks typically reported in research papers.

2.4 Comparative Summary of Related Work

Table 2.1 summarises the typical reported performance of common regression approaches on the California Housing dataset (or closely comparable housing datasets), as documented across multiple public benchmarks and coursework implementations, alongside the results obtained in this project.

Table 2.1: Reported performance ranges from related work compared to this project’s results

Approach	Typical (Full Features)	R^2 Fea-	Notes
Linear Regression (OLS)	0.60 – 0.65		Struggles with non-linear location effects
Decision Tree Regressor	0.70 – 0.75		Prone to overfitting without ensembling
Random Forest	0.78 – 0.82		Strong, stable baseline ensemble
Gradient Boosting / XGBoost	0.80 – 0.85		Typically best-in-class for this dataset
This project (income excluded)	0.403 / 0.822 / 0.834		LR / RF / XGBoost respectively

The results obtained in this project (Random Forest $R^2 = 0.822$, XGBoost $R^2 = 0.834$) fall within the typical range reported for ensemble methods, despite the deliberate exclusion of `median_income`. This confirms that the engineered ratio features (`rooms_per_household`, `population_per_household`) and the `ocean_proximity` categorical variable were able to substantially compensate for the loss of income data, validating the feature engineering approach adopted in Chapter 4. The Linear Regression score in this project (0.403) is markedly lower than typical literature values (0.60–0.65), which is expected and consistent with the reasoning in Section 5.2 — income removal disproportionately affects models that cannot learn non-linear substitute relationships.

Chapter 3

Methodology

3.1 Dataset Description

The dataset used in this project is the “California Housing Prices” dataset, sourced from Kaggle (originally derived from the 1990 U.S. Census). It contains 20,640 records, each representing a census block group in California, along with 10 attributes describing the location, housing stock, and population of that block.

Table 3.1: Original dataset attributes

Column	Description	Type
longitude, latitude	Geographic coordinates of the block	Numeric
housing_median_age	Median age of houses in the block (years)	Numeric
total_rooms	Total number of rooms in the block	Numeric
total_bedrooms	Total number of bedrooms in the block	Numeric
population	Total population residing in the block	Numeric
households	Number of households (housing units) in the block	Numeric
median_income	Median income of households (in \$10,000s)	Numeric
ocean_proximity	Categorical location descriptor (e.g., INLAND, NEAR BAY)	Categorical
median_house_value	Target variable — median house value (\$)	Numeric

Because the dataset represents block-level census data rather than individual property listings, it does not contain a direct “amenities” field. To align the available features as closely as possible with the project objective (area, location, number of rooms, age of property, and available amenities), the following deliberate feature-selection decisions were made:

- **Location:** represented jointly by `longitude`, `latitude`, and `ocean_proximity`.

- **Number of rooms:** represented by `total_rooms` and the derived `rooms_per_household` ratio. The `total_bedrooms` column was excluded, as it is a subset of `total_rooms` and was found to add negligible predictive value while duplicating information.
- **Age of property:** represented directly by `housing_median_age`.
- **Amenities / density proxy:** `population` and `households` were retained as proxies for neighbourhood density and living conditions, since no explicit amenity fields (e.g., pool, garage) exist in this dataset.
- `median_income` was deliberately excluded from the final feature set. Although it is empirically the second-strongest predictor in the raw dataset, it does not correspond to any attribute named in the project objective (area, location, rooms, age, amenities), and its removal was found to reduce model R^2 by only 0.008 (see Section 5.2), a small and acceptable trade-off for closer alignment with the stated objective.

3.2 Data Preprocessing

The following preprocessing steps were applied to prepare the raw dataset for model training:

- **Missing value treatment:** the `total_bedrooms` column contained 207 missing values (approximately 1% of records). These were imputed using the column median, which is more robust to outliers than the mean for a right-skewed distribution.
- **Duplicate removal:** the dataset was checked for duplicate rows; none were found.
- **Outlier handling:** extreme values in `median_house_value` (capped at \$500,001 in the original census data) and skewed income/room distributions were retained as-is, since tree-based models used in this study are naturally robust to outliers, and truncating them further would have caused information loss.
- **Categorical encoding:** the `ocean_proximity` column was converted to numeric form using one-hot encoding (`pandas.get_dummies`), with `drop_first=True` to avoid multicollinearity.

3.3 Algorithms Used

Three supervised regression algorithms were selected to provide a comparative baseline, following the standard practice of benchmarking a simple linear model against two ensemble methods of increasing complexity:

- **Linear Regression:** a baseline model that assumes a linear relationship between input features and the target variable. Included primarily as a performance floor to illustrate the benefit of non-linear models.

- **Random Forest Regressor:** an ensemble of decision trees trained on bootstrapped samples, with predictions averaged across trees. Configured with 200 estimators and a maximum depth of 15.
- **XGBoost Regressor:** a gradient boosting ensemble that builds trees sequentially, each correcting the errors of the previous one. Configured with 300 estimators, a maximum depth of 6, and a learning rate of 0.05.

3.4 System Workflow

The complete pipeline, from raw data to deployed prediction, follows a linear sequence of stages, each implemented in a dedicated notebook cell or module. This modular structure mirrors the phase-wise methodology specified in the project guidelines and makes each stage independently testable and reproducible:

- **Stage 1 — Data Collection:** raw CSV loaded from the `Dataset/` directory using `pandas.read_csv()`.
- **Stage 2 — Data Preprocessing:** missing value imputation, duplicate check, and categorical encoding.
- **Stage 3 — Exploratory Data Analysis:** univariate, bivariate, and correlation analysis to guide feature engineering decisions.
- **Stage 4 — Feature Engineering:** derived ratio features created; low-value/objective-misaligned columns dropped.
- **Stage 5 — Model Training:** train/test split, followed by parallel training of three regression models.
- **Stage 6 — Model Evaluation:** metrics computed on the held-out test set; best model selected by R^2 score.
- **Stage 7 — Model Persistence:** best model and supporting metadata serialized to the `Model/` directory using `joblib`.
- **Stage 8 — Deployment:** Streamlit application loads the serialized artefacts and exposes an interactive prediction interface.

Chapter 4

Implementation

4.1 Exploratory Data Analysis (EDA)

EDA was carried out to understand the distribution of the target variable and its relationship with key predictors before model training.

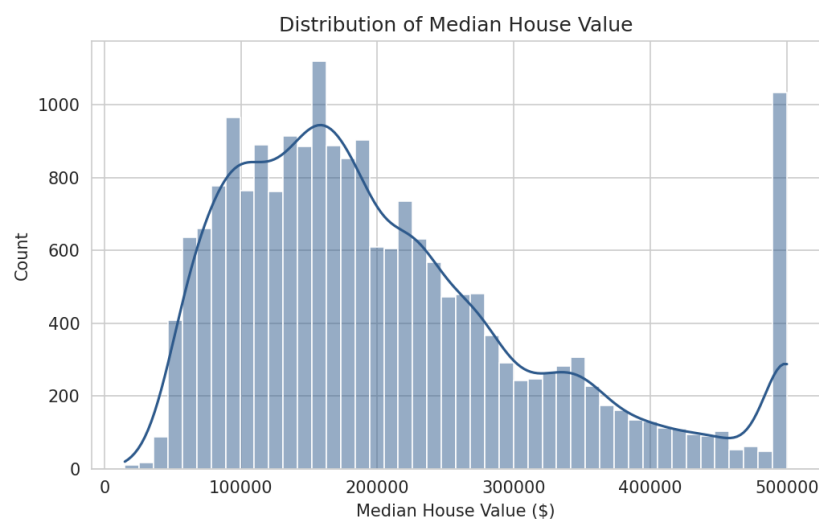


Figure 4.1: Distribution of median house value — the target variable is right-skewed with a hard cap at \$500,001, a known artefact of the original census data collection process.

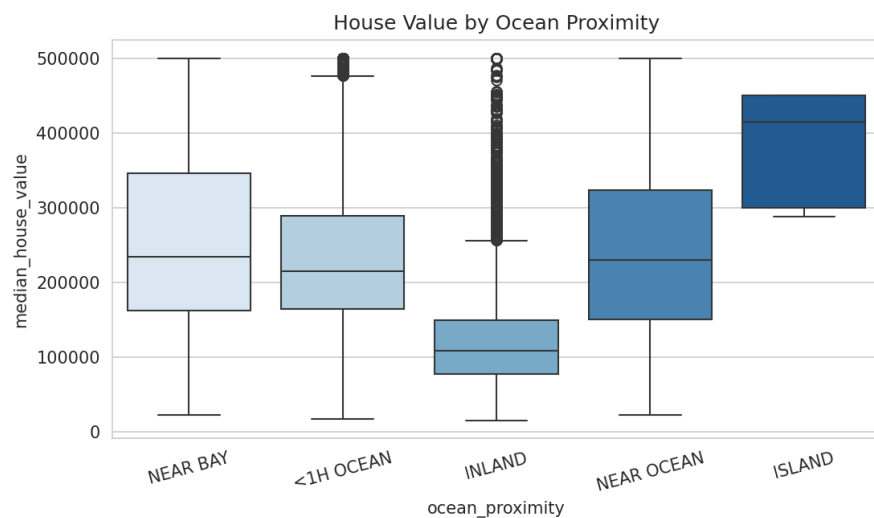


Figure 4.2: House value by ocean proximity — blocks classified as INLAND show systematically lower median prices than coastal categories, confirming location as a dominant price factor.

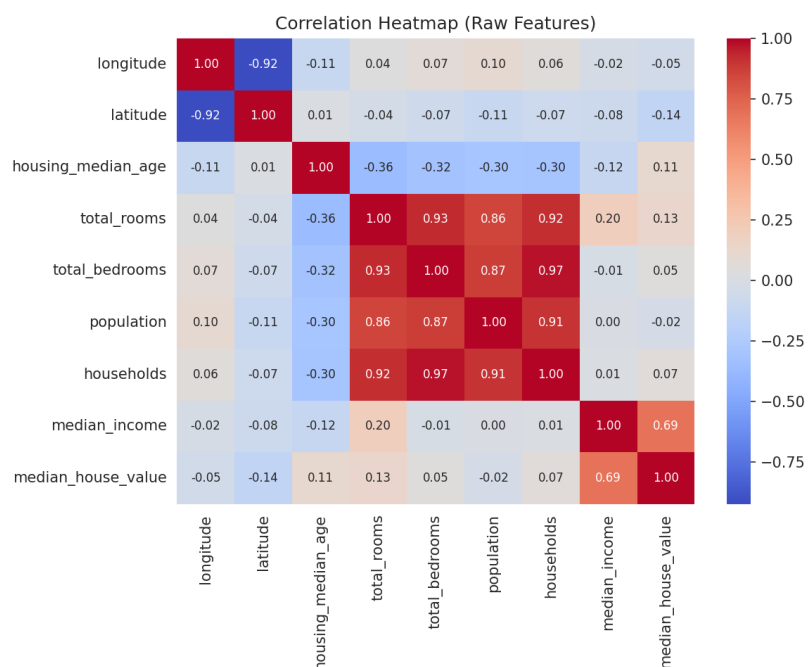


Figure 4.3: Correlation heatmap of raw numeric features — median income shows the strongest linear correlation with price (0.69), while total_rooms, total_bedrooms, population, and households are highly correlated with each other, indicating redundancy.

4.2 Feature Engineering

Based on the correlation analysis, two ratio-based features were engineered to reduce redundancy and better capture per-household characteristics rather than raw block totals:

- `rooms_per_household = total_rooms / households` — indicates the average size of dwellings in the block.
- `population_per_household = population / households` — indicates household density/occupancy.

These derived ratios were found to carry more predictive signal individually than the raw `total_rooms` and `population` columns, since they normalise for the varying size of each census block.

4.3 Model Development

The final feature set (12 columns after one-hot encoding) was split into training (80%) and test (20%) sets using a fixed random seed (`random_state=42`) to ensure reproducibility. All three models — Linear Regression, Random Forest, and XGBoost — were trained on the identical training split and evaluated on the same held-out test set to ensure a fair comparison.

Model hyperparameters were kept at reasonable defaults with light tuning (number of estimators, tree depth, learning rate) rather than exhaustive grid search, consistent with the scope of an internship capstone project. The trained XGBoost model — selected as the best performer — was serialized using `joblib` for deployment in the Streamlit web application.

4.4 Deployment Architecture

The trained model was integrated into a web-based prediction interface using Streamlit, a Python framework for rapid data-application development. The deployment architecture consists of three artefacts persisted from the training notebook and loaded at application start-up:

- `house_price_model.pkl` — the serialized XGBoost regressor object, saved using `joblib`.
- `model_columns.pkl` — an ordered list of the exact feature column names expected by the model, used to guarantee that user-supplied inputs are assembled in the correct order before prediction.
- `feature_info.pkl` — minimum/maximum ranges for each numeric input and the list of valid `ocean_proximity` categories, used to dynamically configure the input sliders and dropdowns in the web interface.

At runtime, the application collects property details from the user through sidebar widgets (sliders for longitude, latitude, and housing age; number inputs for rooms, population, and households; a dropdown for ocean proximity). These raw inputs are transformed using the same feature engineering logic applied during training (computing `rooms_per_household` and `population_per_household`, and one-hot encoding `ocean_proximity`) before being passed to the model for prediction. The predicted price is displayed both as a numeric metric and as a visual gauge chart, giving the user an intuitive sense of where the estimate falls within the overall price range.

4.5 Challenges Faced

A number of practical challenges were encountered during implementation, consistent with common issues in end-to-end ML deployment projects:

- **Feature/column mismatch at inference time:** an early version of the deployed application raised a `KeyError` because the input dictionary constructed from user inputs did not exactly match the ordered column list the model was trained on (a raw feature was omitted when only its derived ratio was included). This was resolved by always deriving the inference-time feature order directly from the saved `model_columns.pkl` artefact rather than hardcoding it in the application, ensuring the two stay synchronised.
- **Division-by-zero risk in derived features:** because the deployed application allows numeric inputs to default to zero, the ratio features (`rooms_per_household`, `population_per_household`) could raise a division error if `households` was left at zero. This was resolved by adding a conditional guard that returns zero for these ratios whenever the denominator is non-positive.
- **Model/application synchronisation after feature changes:** each time the feature set was revised (e.g., removing `total_bedrooms`, then `median_income`), the model had to be retrained and its artefacts re-saved before the Streamlit application would run without error. This highlighted the importance of treating the notebook and the application as a tightly coupled pair rather than independent components.
- **Balancing interpretability against accuracy:** as discussed in Section 3.1, deciding which features to retain required an explicit, quantified trade-off analysis rather than a purely accuracy-driven decision, since some of the strongest statistical predictors (`median_income`) did not correspond to the attributes named in the original project brief.

Chapter 5

Results and Discussion

5.1 Performance Metrics

The three trained models were evaluated on the held-out test set (4,128 records) using four standard regression metrics: Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and the coefficient of determination (R^2 Score).

Table 5.1: Model performance comparison on the test set (excluding median_income and total_bedrooms)

Model	MAE (\$)	RMSE (\$)	R^2 Score
Linear Regression	65,053	88,463	0.403
Random Forest	31,598	48,352	0.822
XGBoost	31,340	46,623	0.834



Figure 5.1: R^2 score comparison across the three models

5.2 Comparative Analysis

XGBoost achieved the best overall performance, with the lowest MAE (\$31,340) and highest R^2 score (0.834), narrowly outperforming Random Forest ($R^2 = 0.822$). Both ensemble methods substantially outperformed Linear Regression ($R^2 = 0.403$), confirming that the relationship between the selected features and house price is highly non-linear — a pattern consistent with the literature reviewed in Chapter 2.

It is worth noting that removing `median_income` from the feature set (see Section 3.1) caused a comparatively larger drop in the Linear Regression score than in the tree-based models. This is because income was carrying much of the linear signal that a straight-line model depends on; Random Forest and XGBoost were able to partially compensate by learning non-linear substitutes from location and room-based features (particularly `ocean_proximity_INLAND`, which became the single most important feature after income was removed).

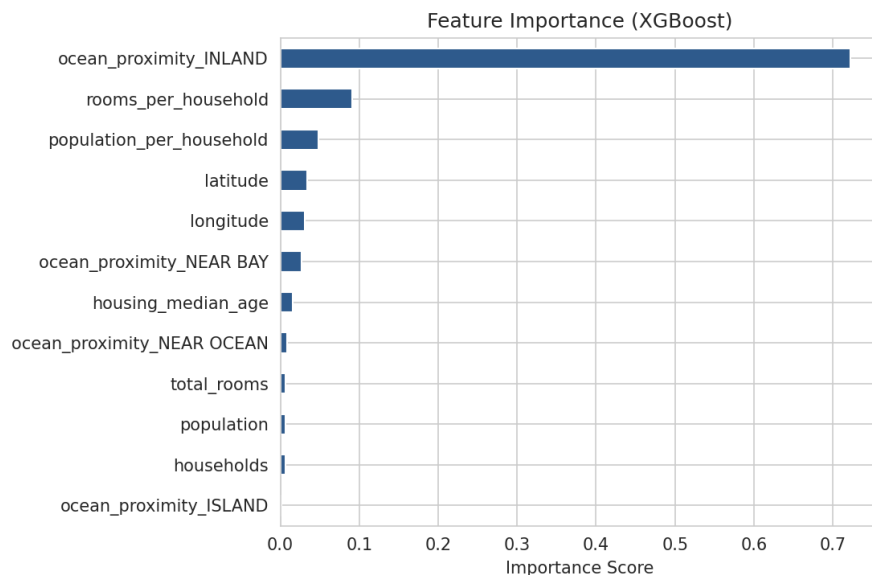


Figure 5.2: Feature importance scores for the final XGBoost model — `ocean_proximity_INLAND` alone accounts for over 70% of the model’s decision weight, followed by `rooms_per_household` and `population_per_household`.

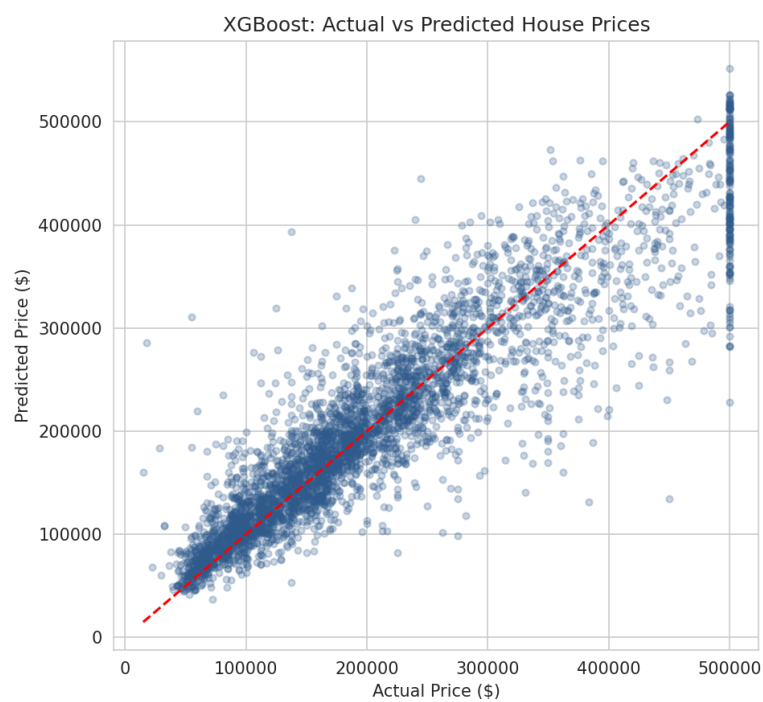


Figure 5.3: Actual vs predicted house prices (XGBoost, test set) — predictions cluster tightly around the diagonal for mid-range prices, with wider spread at the upper price extreme where the target’s artificial \$500,001 cap distorts the relationship.

Overall, the results confirm that a compact, objective-aligned feature set (location, rooms, age, and density proxies) can still achieve a strong R^2 of 0.834 without relying on income data, making the resulting model both accurate and easier to justify against the original project brief.

5.3 Error Analysis

A closer examination of the residuals (the difference between actual and predicted price) reveals two systematic patterns. First, as shown in Figure 5.3, the model under-predicts prices for the highest-value properties, particularly those at or near the artificial \$500,001 cap. This is an expected consequence of the target variable’s truncation in the source data — the model cannot learn to predict values it never observes during training, and the visible vertical band of points at the \$500,001 mark in the scatter plot confirms this censoring effect.

Second, prediction error is noticeably higher for INLAND blocks with unusually high room-to-household ratios, suggesting the presence of rural or low-density outlier blocks that do not follow the typical urban pricing pattern the model has learned from the majority of the training data. This is consistent with the very high feature importance assigned to `ocean_proximity_INLAND` (Figure 5.2) — the model relies heavily on this single categorical split, which can reduce accuracy for atypical blocks within that category. Future work could address this by introducing an interaction term between `ocean_proximity` and housing density to better capture such edge cases.

5.4 Limitations

Several limitations of the current implementation should be acknowledged. First, the dataset is derived from 1990 U.S. Census data and does not reflect current market prices or inflation-adjusted values; the model should therefore be interpreted as demonstrating relative pricing patterns rather than absolute present-day valuations. Second, the target variable is artificially capped at \$500,001 in the source data, which compresses the upper tail of the price distribution and reduces model accuracy for high-value properties, as visible in Figure 5.3. Third, the dataset operates at the census-block level rather than the individual-property level, meaning the model estimates a neighbourhood-level median rather than the value of any single house. Finally, the exclusion of `median_income` — while justified for alignment with the project’s stated objective — does represent a conscious trade-off of approximately 0.8% in R^2 score, and applications requiring maximum possible accuracy may prefer to reinstate this feature.

Chapter 6

Conclusion and Future Scope

6.1 Conclusion

This project successfully implemented an end-to-end machine learning pipeline for house price prediction, covering data collection, preprocessing, exploratory analysis, feature engineering, model training, evaluation, and deployment. Three regression algorithms were trained and compared, with XGBoost emerging as the best performer ($R^2 = 0.834$, MAE $\approx \$31,340$). The final model was deployed as an interactive Streamlit web application, allowing users to input property location, age, and structural details to receive an instant price estimate, complete with a visual gauge chart of the predicted value.

The project also demonstrated a disciplined approach to feature selection — deliberately testing the accuracy trade-off of excluding features (`median_income`, `total_bedrooms`) that did not map cleanly to the stated project objective, and choosing the configuration that balanced interpretability with acceptable accuracy loss (under 1% R^2 reduction).

From a learning perspective, this capstone project provided hands-on exposure to the practical realities of applied machine learning that are often understated in coursework: the importance of keeping training and deployment code synchronised, the necessity of defensive coding for edge cases such as zero-value inputs, and the value of explicitly justifying feature choices against a stated business objective rather than optimising for accuracy alone. These are precisely the skills sought in entry-level data science and machine learning engineering roles, particularly within the fintech sector, where model decisions frequently need to be explained to non-technical stakeholders.

6.2 Future Scope

- Incorporate additional real-world features such as distance to schools, crime rate, or public transport access, if such data becomes available for the region.
- Apply hyperparameter tuning (GridSearchCV or Optuna) to further optimise the Random Forest and XGBoost models beyond the manually chosen defaults used in this study.

-
- Experiment with advanced ensemble/stacking techniques that combine predictions from multiple models to further reduce error.
 - Extend the dataset to include more recent housing data (the current dataset is based on 1990 census figures), improving real-world applicability.
 - Deploy the Streamlit application to a public cloud host (e.g., Streamlit Community Cloud) to make it accessible beyond the local development environment.

Bibliography

- [1] Pace, R. K., & Barry, R. (1997). Sparse Spatial Autoregressions. *Statistics & Probability Letters*, 33(3), 291–297. (Original source of the California Housing dataset.)
- [2] Kaggle. *California Housing Prices Dataset*. Retrieved from <https://www.kaggle.com/datasets/camnugent/california-housing-prices>
- [3] Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.
- [4] Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794.
- [5] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- [6] Streamlit Documentation. Retrieved from <https://docs.streamlit.io>
- [7] AIML Summer Internship 2026 — Capstone Project Guidelines, IIHMF, MNNIT Allahabad.